

I heard that in slice, we mutate the state. But why this code is returning something in reducers?

```
const userSlice = createSlice({
  name: "user",
  initialState: null,
  reducers: {
    addUser: (state, action) => {
      return action.payload;
    },
    removeUser: (state, action) => {
      return null;
    },
  },
});
```

In **Redux Toolkit (RTK)**, you *can* mutate state inside reducers **or** you can return a new state — **both are valid**.

This is because RTK uses **Immer** under the hood.

Why mutation is allowed

In normal Redux, reducers must be pure functions and must **not mutate state**. But RTK wraps your reducer logic in the **Immer** library, which:

- lets you *write* mutations (e.g., `state.value = 5`)
- but converts them into **immutable updates** internally

So this is valid:

js

 Copy code

```
addUser(state, action) {
  state.name = action.payload.name;
}
```

You're *mutating*, but Immer creates a **new immutable state object**.

Why your reducers are returning something

Immer also allows you to **return a completely new state** instead of mutating.

If you return something:

Immer uses your return value as the **next state**, ignoring the draft.

So this:

```
js
```

[Copy code](#)

```
addUser: (state, action) => {  
  return action.payload;  
},
```

says: "Replace the user state entirely with action.payload."

This:

```
js
```

[Copy code](#)

```
removeUser: () => {  
  return null;  
}
```

says: "Replace the user state with null."

Both are completely valid patterns.

Key rule

Inside a `createSlice` reducer, you can **do either one**:

1. Mutate the state

(becomes immutable automatically)

2. Return a new value

(replaces state completely)

But **not both in the same reducer**, because returning overrides any mutations.

✓ Summary

- RTK uses Immer → reducers **can mutate or return**.
- Returning a value replaces state.
- Mutating modifies the draft which Immer turns into immutable state.

Your slice works because you are **replacing the whole user state** instead of mutating its properties — totally valid and common for auth/user slices.

If you want, I can show you an example of both mutation and return styles side-by-side.

